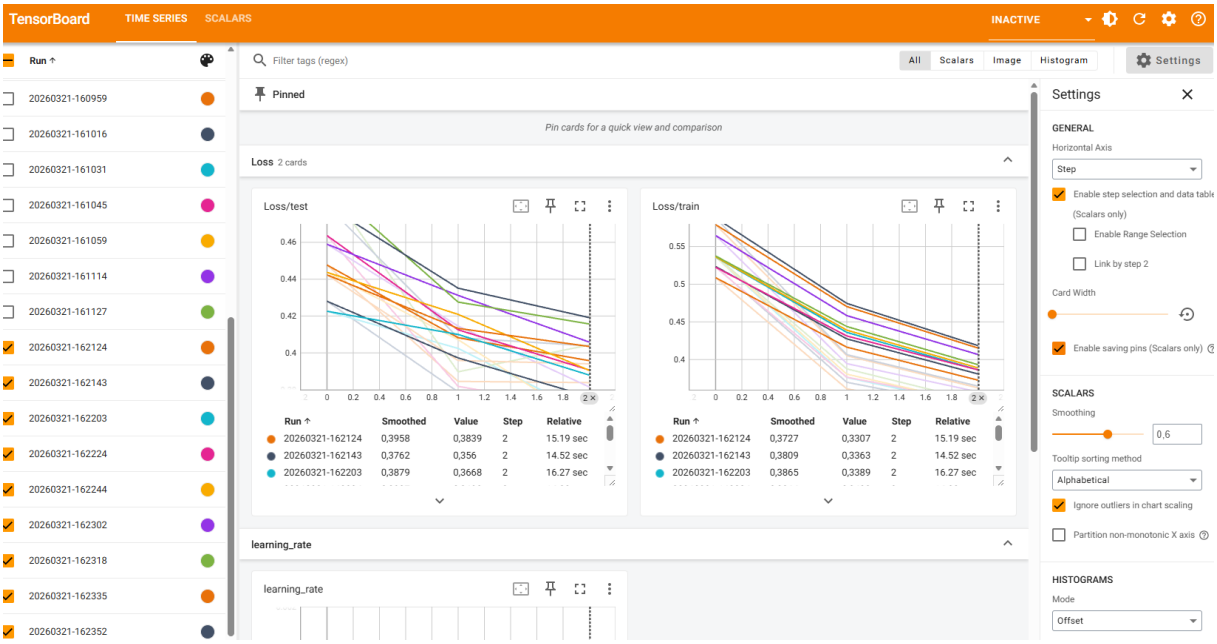
Before experimenting with the hyperparameters, I have taken a screenshot of the base run with the standard parameters from the example script. See screenshot below.



There are 9 runs visible, all with a steady decline in training loss. The decline in test loss is varying a bit more, although all of them seem to be sloping downwards, hence indicating improvement in all of the runs. In the baseline experiment, the smoothed version of the following runs was the best (both for training and test):

Run	Smoothed	Value	Step	Relative
20260321-162143	0,3762	0,356	2	14.52 sec

Run	Smoothed	Value	Step	Relative
20260321-162124	0,3727	0,3307	2	15.19 sec

The 162143 run has the following connected model settings:

College1 - NN > modellogs > 20260321-162143 > model.toml

```toml
[model]
training = true
num_classes = 10
units1 = 256
units2 = 128

[types]
training = "bool"
num_classes = "int"
units1 = "int"
units2 = "int"
```

And the 162124 run has the following:

College1 - NN > modellogs > 20260321-162124 > model.toml

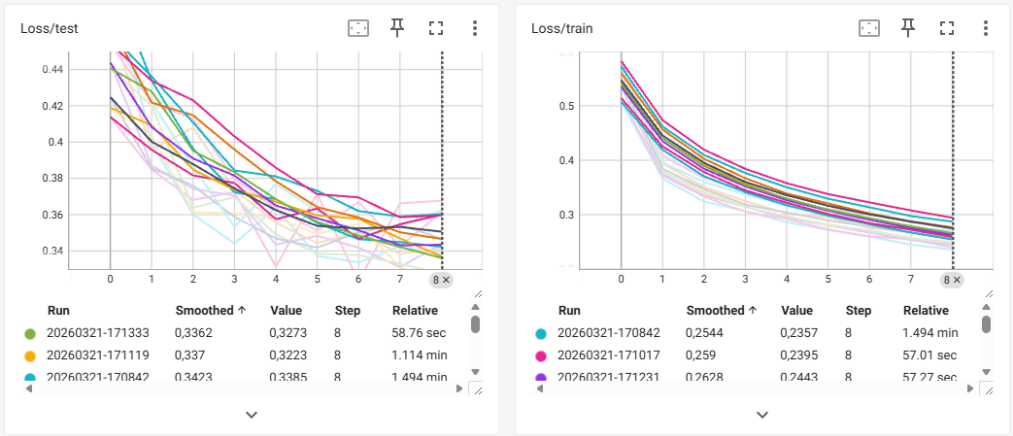
```toml
[model]
training = true
num_classes = 10
units1 = 256
units2 = 256

[types]
training = "bool"
num_classes = "int"
units1 = "int"
units2 = "int"
```

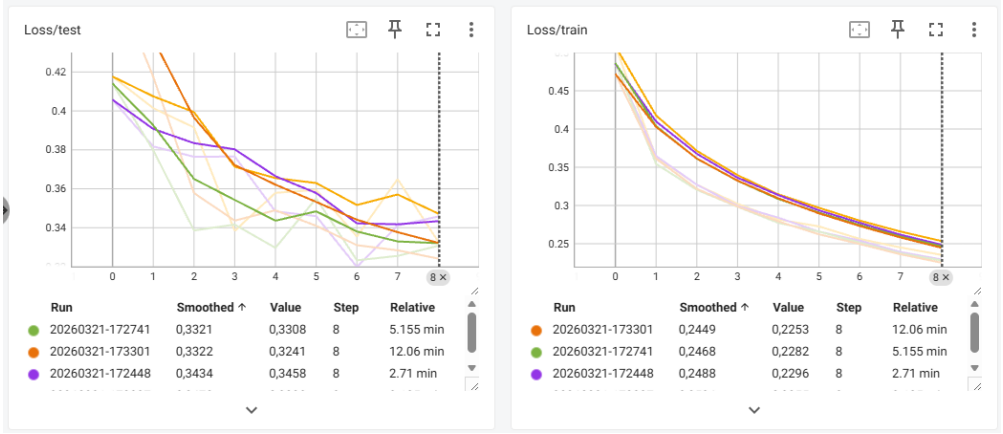
My hypothesis is as follows:

I expect the training and test loss to improve when I increase the amount of epochs and increase the amount of units. I will not increase the amount of layers (yet) since I don't want to risk overfitting. I think a lot of improvement is already possible through increasing the epochs and the unit size. Also, especially since both training and test loss lines are declining, it looks like the model is still busy learning. So I will start with increasing only the epochs to evaluate the influence.

In my first test, I increased the amount of epochs to 9. The training obviously took longer due to increasing the epochs, but the results also increased a lot. What is mainly interesting to see is that my lines are becoming more flat, so it looks like increasing the epochs took care of the fact that the model was not done with its steep learning yet. That's resolved now!

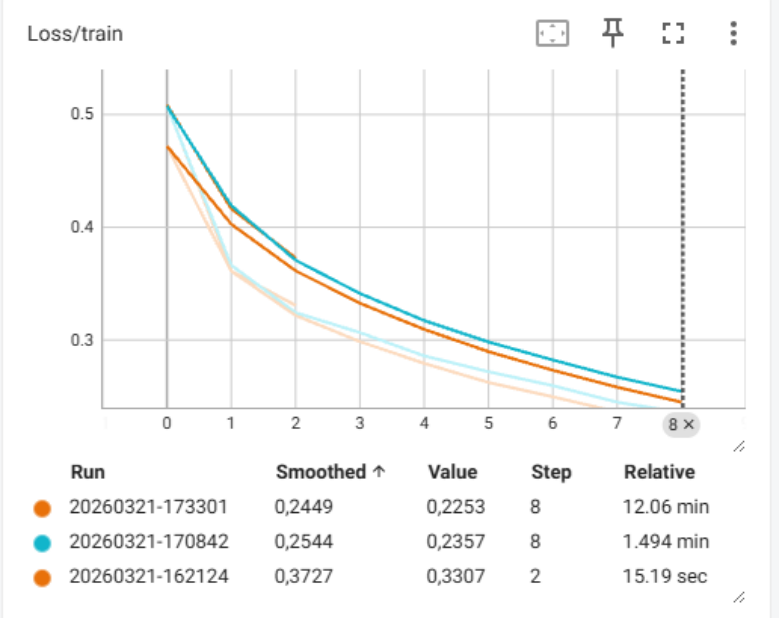


For the final experiment, I will increase the amount of units and keep the amount of epochs at 9. Again, training took way longer due to adding 1024 in units. It actually took so long that I decided to remove 64 and 128 and keep it at 256 and 1024 (since 256 seemed to be better than the other units in previous runs). So I kept epochs at 9 and added 1024 to increase model performance and deleted the other units to improve training time a bit. The screenshot below shows that training and test loss both improved when compared to previous run.



For comparison, I have added below the graphs of the training and test loss of the best models of each adjustment including the baseline, so that it's clearly visible how changing the parameters helped improve model performance.

First, the training loss. The training loss seems to improve with every iteration change. Although the last iteration change is probably not worth it. Training time went up by almost a factor of 12 while training loss only slightly improved.



Lastly, the test loss. Although the test loss improved, the change in model performance is relatively low compared to the extra time it took to train the model. From this perspective, its probably not worth it to train the model 5 times longer for such a small improvement. A potential gain is better found in tuning other parameters (but of course this depends on the context of the situation at hand).